Phase 5 developer handoff

Architecture

`backend/app/modernization/` contains the planning subsystem:

Module	Responsibility
`engine.py`	Pure deterministic findings, scores, decisions, target alternatives, dependency waves and planning graph
`validation.py`	Independent publication checks for references, arithmetic, conflicts, supported conclusions, dependency ordering and no execution
`service.py`	Version-bound materialization, fingerprints, immutable persistence, review revisions, graph recovery and typed explanations
`schemas.py`	Strict bounded proposal, review and question contracts
`models.py`	Tenant-owned assessments and append-only planning review records
`routes.py`	Authenticated API and runtime-evidence permission inheritance

`frontend/src/modernization/PlanningWorkspace.tsx` provides a distinct navigation entry. Existing static modernization scores and runtime interfaces remain available. Phase 5 does not replace the scanner, runtime receiver, source gateway or model gateway.

Persistence and reproducibility

Migration `0006` adds `modernization_assessments` and `modernization_reviews`. It adds no new user, credential, policy or provider record and does not rewrite existing analysis/evidence data.

An assessment stores its actor, version ID, fingerprint and frozen JSON document. The fingerprint covers the captured version fingerprint, canonical document/facts/evidence digest, objective/strategy, rule version and calibration version. The UUID is namespaced by tenant plus fingerprint. Unique tenant/fingerprint prevents duplicate persisted proposals.

Before generation, the source analysis JSON digest must match the immutable version manifest. Changed source snapshots yield 409. Reviewed domain memberships are taken from the captured version, not a mutable UI state. Source snapshots and canonical Phase 4 graphs are never modified by planning.

Rule version: `modernization-intelligence-1`. Calibration version: `fixture-calibration-1`. Validator version: `proposal-validator-1`. Increment derivation versions when rule behavior changes so historical proposals are not reused as new results.

The service currently uses the existing in-process evidence lock. Deployment remains single API process. Unique database constraints are a second integrity boundary, not a distributed task/lease implementation.

Scores and uncertainty

For unreviewed versions, reuse `static.modernization` structural scores, component weights and components exactly. For reviewed memberships, reuse `enrich_domains` and `assess_and_rank` over a copied static class graph, updating cycle impacted domains. No source or historical result mutation occurs.

Runtime readiness reuses the existing transparent component convention:

```
cohesion = 100 * observed internal mapped calls / all outgoing mapped calls
cycles = 0 if the domain participates in an observed cross-domain SCC, else 100
persistence = 25 if a shared observed resource candidate exists, else 50 (unknown)
runtime = 0.60 * cohesion + 0.25 * cycles + 0.15 * persistence
combined = 0.60 * structural + 0.40 * runtime
```

No mapped outgoing calls means runtime `null` and combined equals structural. Cycle absence is sample-scoped, not proof of acyclicity. Coverage does not scale readiness. Runtime call counts come from canonical observed facts, not client fields or model output.

Confidence is a separate 0–100 heuristic evidence index:

Component	Weight	Input
Static support	.25	100 for a valid captured static baseline
Runtime observation	.25	Domain observed-class percentage, zero without selected runtime
Mapping/ingestion quality	.15	100 minus bounded diagnostic penalties, zero without runtime
Boundary review	.15	100 only for a captured explicit domain review
Behavioral validation	.10	0: unavailable in Phase 5
Data ownership	.10	0: unavailable in Phase 5

Quality subtracts 20 points per missing parent, unresolved node, dropped/conflicting span, invalid parent link or binding rejection, capped at 100. Any ingestion warnings or unverified spans impose at least a 25-point penalty. Diagnostics are global to selected sessions and conservative, not domain-specific statistical error estimates. Quality gaps add a review blocker. Labels are LIMITED below 50, otherwise PARTIAL; Phase 5 never labels the evidence fully validated.

Calibration fixtures assert static-only confidence 25, full observed/reviewed confidence 80, and complete quality penalty confidence 65 while readiness remains unchanged. These are arithmetic/rule acceptance fixtures, not expert-reviewed empirical success calibration. Real-world calibration against representative modernization outcomes remains future work.

Decision precedence and findings

Each domain has evidence-linked findings and one decision, selected in this order:

1. Conflicting facts or cloud/upgrade strategy: `MANUAL_REVIEW_REQUIRED`.
2. Cross-domain SCC in the combined static/runtime dependency graph: `BREAK_CYCLE_FIRST`.
3. Shared observed database resource: `ISOLATE_DATA_FIRST`.
4. Structural cross-domain ratio above .5: `REDUCE_COUPLING_FIRST`.
5. Class observation below 80%, mapping/quality gaps or unreviewed domain: `COLLECT_MORE_EVIDENCE`.
6. Otherwise modular target: `RETAIN_MODULAR_MONOLITH`; distributed target: `PREPARE_FOR_EXTRACTION`.

Database resource equality requires system, server, namespace and resource, with direct mapped callers in multiple domains. It establishes a sharing candidate only. Missing resource identity never establishes independence. Source/build, behavioral contracts, data ownership and human change approval remain prerequisites on every proposal. `EXTRACT_NOW` cannot be published in Phase 5.

Cycle facts, high coupling, shared resources, evidence conflicts, unresolved boundaries, observation gaps, unreviewed boundaries and unknown behavior/data generate typed findings. Each links to domain facts and/or canonical evidence; no free-form model conclusion can enter this publishing path.

Independent validation

The validator checks fact evidence existence, endpoint correspondence and evidence-kind classification; conclusion references; bounded finite score arithmetic and normalized weights; no invented behavioral/data confidence; finding/decision domain consistency; conflict and cycle blockers; no unsupported optimistic decisions; dependency-first SCC waves; and graph endpoint integrity.

Failure yields HTTP 409 and no assessment is persisted. The validator checks a planning proposal, not customer behavioral equivalence, security of generated code or empirical suitability. Clients cannot submit generated findings, scores, Cypher or source patches.

Targets, waves and review

Six target strategies are supported as proposals. Modular monolith assigns in-process deployment; other strategies leave deployment undecided. Interfaces reflect observed/static domain dependencies with unspecified contracts. No business capability, REST schema, event or transaction contract is fabricated. Cloud/Java/framework proposals explicitly block on compatibility inventory.

Waves use the SCC condensation DAG, selecting dependency leaves before consumers. All domains in a cyclic group remain in the same wave. Wave metadata includes domains, dependencies, readiness, confidence, difficulty, blockers, prerequisites and unmeasured expected impact.

Review records are immutable inserts with sequential revision, actor, note and state. The expected revision rejects stale browser submissions. Reviews are limited to 100 per assessment. The latest captured application version defines CURRENT; older versions become SUPERSEDED and cannot receive new reviews. An approved plan remains blocked for code execution. Revised objectives/strategies produce new immutable proposals; changed boundaries/evidence require new architecture versions.

Graph and explanation

Reuse the existing fixed parameterized Neo4j projection/query adapter under a separate `tenant:assessment_id` scope. All node IDs are assessment-namespaced, including canonical fact/evidence reference nodes. This avoids collisions with the Phase 4 version graph. Reference nodes retain canonical IDs; decision-to-finding-to-fact/evidence relationships form the explanation chain. Relationships include SUPPORTS, REQUIRES, BLOCKED_BY and PRECEDES.

Creation commits PostgreSQL first, then attempts projection if enabled. Projection errors store exception class only. Graph reads verify marker digest and counts; rebuild retries idempotently. Planning retries are manual in this release. Review events and current-context status are authoritative relational data and are not copied into the immutable planning graph.

Typed WHY/BLOCKERS/NEXT_STEP questions retrieve a saved decision and its canonical references without an LLM. Unsupported question values are rejected. The existing AI gateway is untouched. Generative narrative, arbitrary graph queries and code execution are outside this workflow.

API

Base: `/api/v1/architecture-versions/{vid}/modernization`.

Method/path	Behavior
POST `/preview`	Validate and return a proposal without saving
POST base	Idempotently save an immutable proposal
GET base	Up to 100 proposals for this version

Method/path	Behavior
GET `/{aid}`	Frozen document, live context, review history and projection status
GET `/{aid}/export`	Frozen proposal only; unaffected by reviews or projection failures
POST `/{aid}/review`	Append review with expected revision
POST `/{aid}/explain`	Typed evidence-linked answer
GET `/{aid}/graph`	Bounded verified graph; optional scoped root
POST `/{aid}/projection/rebuild`	Retry isolated planning projection

ANALYSIS_READ protects reads and explanation requests. ANALYSIS_WRITE protects preview/save/review/rebuild. RUNTIME_READ is additionally required for every operation on a version containing runtime. Scoped database access and explicit version/assessment ownership checks prevent tenant and version substitution. Existing CSRF, audit-intent recording and 2 MiB version-route body cap apply. Proposal/objective and review note lengths are bounded. No new external destination is introduced.

Planning caps: 500 domains, 30,000 facts, 100,000 graph nodes, 250,000 graph edges; graph queries show at most 500 nodes and 1,000 induced edges. This is not a large-repository performance certification.

Verification and extension

Run `backend/tests/test_modernization.py`; opt into actual PostgreSQL/Neo4j tests with `ARCHINTEL_RUN_POSTGRES_TESTS=1` and `ARCHINTEL_RUN_GRAPH_TESTS=1`. Real integration imports the retained Phase 3 PetClinic capture into a disposable PostgreSQL schema, verifies runtime-aware planning and restart preservation, then removes its schema and graph scopes. It does not execute a fresh Java workload or alter the user's workspace.

Browser workflow: `frontend/e2e/04-modernization.spec.ts` using `playwright.security.config.ts` and isolated port 18001. See verification for actual final results and integration report for measured output.

Next phase should add pinned source/build identity, behavioral contracts and safe task/approval/rollback primitives. Do not remove current extraction blockers merely to demonstrate an executable action. Add evidence-backed prerequisites and independent checks before enabling any transformation.